

main() method :

It is a user-defined non static method because a user is responsible to define some logic inside the main method.

main () method is very important method because every program execution will start from main () method only, as well as the execution of the program ends with main () method only.

JVM internally create the object for compiler generated class to call this java 25 non static main method.

IO class :

It is predefined utility class (Helper class) available in java.lang package (library) from JDK 25V onwards.

It is used to perform **input & output** operation in java.

It has provided various static methods (only static methods) to perform input & output operation.

1) public static void println(Object obj)

It is a predefined static method of IO class.

It is used to print the message or data on the console window and after printing it will move the cursor to the next line.

2) public static void print(Object obj)

It is a predefined static method of IO class.

It is used to print the message or data on the console window and after printing it will keep the cursor in the line.

WAP to add two numbers in java :

Addition.java

```
void main()
{
    int x = 100;
    int y = 200;
    int z = x + y;
    IO.println(z);
}
```

Note : Output is 300 but It is not a user-friendly message

How to provide user-friendly message in Java :

```
void main()
{
    int x = 200;
    int y = 300;
    int z = x + y;
    IO.println("Addition is :",z); //IO.println(String, int); //Compilation error
}
```

Note : In IO class there is no predefined method called println(String , int) which is accepting String, int as a parameter

* In order to provide user-friendly message, we can use '+' operator.

Behaviour of '+' operator :

While working with '+' operator, if any of the operand (left side OR right side) is **String** type then this '+' operator will behave as a **String concatenation operator**.

Example :

1 + 1 = 2 [Here + is working as Arithmetic Operator]

"2" + 2 = 22 [Here + is working as String concatenation Operator]

3 + " Hello" = 3 Hello [Here + is working as String concatenation Operator]

"Sum is "+ 300 = Sum is 300 [Here + is working as String concatenation Operator]

int z = 900;
"Sum is "+z = Sum is 900 [Here + is working as String concatenation Operator]

WAP to add two numbers with user-friendly message :

Sum.java

```
void main()
{
    int x = 200;
    int y = 300;
    int z = x + y;
    IO.println("Addition is :"+z);
}
```

//WAP to add two numbers without using 3rd variable

```
void main()
{
    int x = 100;
    int y = 200;
    IO.println("Sum is :"+x+y); //Sum is 100200
    IO.println(+x+y); //300
    IO.println(""+x+y); //100200
    IO.println("Addition is :"+(x+y)); //Addition is : 300
}
```

```
IQ :
IQ.java
void main()
{
    String str = 90 + 90 + " Java " + 40 + 40;
    IO.println(str); //180 java 4040

    //String y = 100 + 100 + "Java" + 90 - 40; //error
    //IO.println(y);
}
```

Tokens in java:

Token is the **smallest unit** of the program which is identified by the compiler.

Without token we can't complete statement or expression in java.

Token is divided into 5 types in java

- 1) Identifiers
- 2) Keywords
- 3) Literals
- 4) Punctuators (Separators)
- 5) Operators

Identifiers:

A **name** in java program by default considered as identifiers.

Assigned to variable, method, classes to **uniquely** identify them.

Example

```
class Fan
{
    int coil ;

    void switchOn()
    {
    }
}
```

Here Fan (Name of the class), coil (Name of the field) and switchOn(Name of the Method) are identifiers.

Rules for defining an identifier:

- 1) Can consist of uppercase(A-Z), lowercase(a-z), digits(0-9), \$ sign, and underscore (_)
- 2) Begins with letter, \$, and _ [cannot start with number]
- 3) It is case sensitive
- 4) Cannot be a keyword
- 5) No limitation of length

Keywords :

A keyword is a predefined word whose meaning is already defined by the compiler.

In Java, all keywords must be written in **lowercase** only.

We cannot use a keyword as an identifier.

true, false, and null look like keywords, but actually they are **literals**.

As of now, Java has 67+ reserved words (including keywords, literals, and reserved words).

goto and const are reserved words in Java.

Literals :

Any constant which we are assigning to field/variable is called Literal.

In java we have 5 types of Literals:

- 1) Integral Literal
 - 2) Floating Point Literal
 - 3) Boolean Literal
 - 4) Character Literal
 - 5) String Literal
- Note : null is also a literal

Integral Literal :

* If a numeric literal does not contain decimal OR fraction then It is called Integral Literal.

Example : 12, 90, 234, 1890

* In Integral literal we have 4 data types

- a) byte (8 bits)
- b) short (16 bits)
- c) int (32 bits)
- d) long (64 bits)